

Preguntas final base de datos

Generales

1. ¿Cuál es el elemento más importante de una arquitectura DBMS?

El elemento más importante de una DBMS es el optimizador ya que obtiene información estadística de la base de datos, para evaluar cual es la mejor estrategia para realizar la consulta, generando un plan de ejecución que consiste en un plan detallado para la generación de consultas.

2. Explique diferencias entre el concepto de integridad y consistencia.

Consistencia: En aquellos casos donde la redundancia de los datos no es nula, las DBMS deberán garantizar que cualquier modificación a un dato redundante se actualizara en todas sus copias

Integridad: Los DBMS deberán no solo abstraer al usuario de cómo se almacenan físicamente los datos, sino que también deben garantizar que los datos no sufrirán modificaciones que no sean intencionales.

3. ¿De qué elementos o artefactos se sirve el generador de código para producir el código?

El generador de código se servirá de la información provista por el runtime processor para generar código que puede almacenarse para luego ejecutarse (en el caso de consultas pre compiladas), o para ser interpretado inmediatamente y también recibe el plan de ejecución provisto por el optimizador.

4. ¿Por qué no hay un operador “DISTINCT” en algebra relacional?

Como el algebra relacional está fuertemente relacionado con la teoría de conjuntos, dentro de una relación (o el resultado de una proyección, por ejemplo) no puede haber tuplas repetidas.

Almacenamiento e Índices

1. ¿Cuántos índices tipo cluster (agrupados) puede tener una tabla? Justificar

Los índices clúster ordenan y almacenan las filas de los datos de la tabla o vista de acuerdo con los valores de la clave del índice. Son columnas incluidas en la definición del índice. Solo puede haber un índice clúster por cada tabla, porque las filas de datos solo pueden estar almacenadas de

una forma.

2. ¿Qué condición debe cumplir algún/os atributos para que pueda aplicarse un índice Bitmaps? Dar un ejemplo

Bitmaps son arreglos con cadenas de bits que representan, para cada ítem de datos, cual es el valor del dominio de la clave por la cual se hizo el índice que tiene el atributo. Suele recomendarse utilizarlo para atributos cuyo dominio tenga una cardinalidad baja (por ejemplo, genero podría tener cardinalidad 2). No tienen buen comportamiento en inserciones/eliminaciones. En los índices tipo bitmap se generan cadenas de bits a partir de los atributos que se desean usar para indexar. Cuando se esté realizando una comparación entre un valor y la cadena de bits (índice) lo que se hace es generar la cadena de bits del valor a comparar y se aplica la función AND. Si el resultado devuelve un 1, este habrá matcheado correctamente. No se recomienda para aquellas relaciones que presenten cambios frecuentes porque habría que generar la cadena de bits continuamente.

Cardinalidad baja

M	F
1	0
1	0
0	1
1	0

idcli	nombre	sexo	categoría
112	José	M	3
115	Ramón	M	5
119	Susana	F	5
112	Guillermo	M	4

1	2	3	4	5
0	0	1	0	0
0	0	0	0	1
0	0	0	0	1
0	0	0	1	0

Transacciones y concurrencia

1. Describa algún método para el tratamiento de Deadlocks que permita actuar detectando deadlocks en la base de datos utilizando grafos como estructura principal de análisis en tiempo real.

Ocurre deadlock cuándo dos transacciones simultáneas están esperando que la otra libere su candado sobre un elemento. Voy a mencionar dos métodos que no necesitan utilizar Timestamps. Está el No Waiting que, si una transacción no puede obtener un candado, se aborta de inmediato y se reinicia después de cierto tiempo aleatorio sin comprobar si ocurrirá o no un deadlock. Obviamente este enfoque puede hacer que muchas transacciones se aborten innecesariamente. Una forma sencilla de detectarlos es construyendo un grafo de espera:

- Se crea un nodo por cada transacción del schedule (plan)
- Se crea un arco de T_i a T_j si la T_i está esperando para bloquear un

elemento X que está bloqueado momentáneamente por Tj.

- Se elimina el arco anterior cuándo Tj libera el/los candado/s sobre el/los elementos que estaba esperando Ti.
- Si en el grafo hay un ciclo, quiere decir que hay un estado de deadlock.

2. Técnicas de recuperación. ¿Qué diferencias hay entre las técnicas de recuperación basadas en actualización diferida respecto de las de actualización inmediata?

En la técnica de recuperación basada en actualización diferida, al interactuar transaccionalmente con un DBMS no se graban los datos directamente al almacenamiento físico, sino que se escribe en un log al momento de la confirmación de la transacción, y luego de manera diferida a los archivos físicos de datos al momento del checkpoint del log transaccional. En estos casos, si una transacción falla antes de llegar a su confirmación (commit), no hay necesidad de deshacer ninguna operación, dado que la transacción no ha afectado la base de datos en ningún modo.

En la técnica de recuperación basada en actualización inmediata, al interactuar transaccionalmente con un DBMS se graban los datos directamente al almacenamiento físico. Sin embargo, también se utiliza un log transaccional, a fin de que el DBMS sea capaz de recuperarse en caso de fallos. Estas variantes hacen uso de un mecanismo de escritura anticipada en el log transaccional. En estos casos, las medidas a tomar ante una recuperación tienen que ver con deshacer las escrituras realizadas en la base de datos.

3. Las propiedades ACID conforman un conjunto de características muy deseables en muchos sistemas ¿Por qué motivo el cumplimiento estricto de las propiedades ACID puede afectar el escalamiento o la performance de un RDBMS en ambientes distribuidos?

El punto principal de realizar transacciones distribuidas es que se garanticen las propiedades ACID de las transacciones. Sin embargo, esto demanda más recursos que logren garantizarlos afectando directamente a la performance en ambientes distribuidos. Se requiere más sistemas de control y mensajes que logren garantizar, por ejemplo, la propiedad de aislamiento, para que las transacciones estén libres de problemas de concurrencia.

4. ¿Cuándo ocurre el efecto conocido como inanición en los protocolos de prevención de deadlocks?

Podemos detectar deadlocks. Lo que hacemos es verificar periódicamente si el motor está en un estado de deadlock. Para detectar deadlocks usamos grafos. Donde hay un ciclo, hay un deadlock. Si hay deadlock, el DBMS deberá elegir que transacción o transacciones deberá abortar o reiniciar. Estas transacciones son conocidas como transacciones víctimas. Ocurre el efecto conocido como inanición cuando la transacción víctima es siempre la misma.

5.

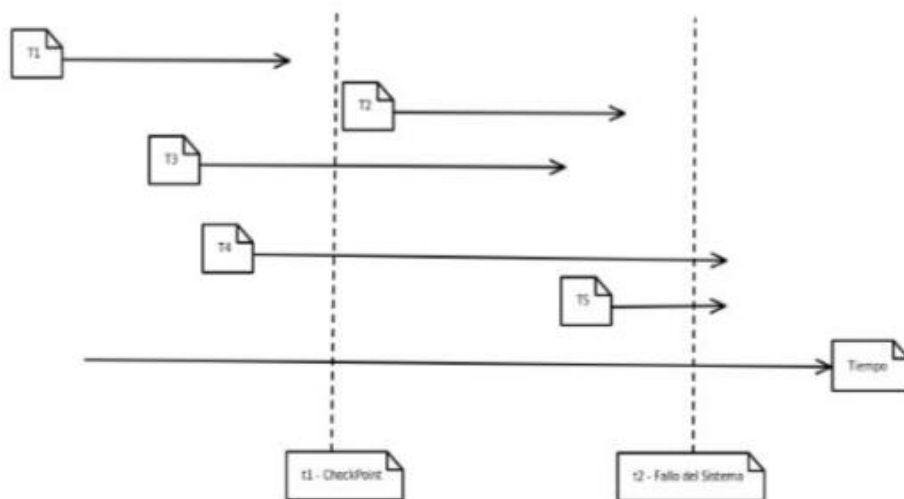
Dado el siguiente Schedule transaccional donde se produce un deadlock: **Transaccion**

T1	T2
S(a)	S(b)
R(a)	R(b)
X(b)	X(a)
Free(a)	Free(a)
R(b)	R(a)
W(b)	W(a)
Commit	Commit

Que acción tomaría un DBMS que opere con la técnica Cautious Waiting suponiendo que la primera transacción en pedir un lock ya adquirido es la T1.

- Aborta T2 y T1
- Aborta T1 y T2
- Aborta T2
- Aborta T1, ya que se queda esperando a T2, pero luego de cierto tiempo de espera se aborta**
- Espera a la liberación de los candados de manera cautelosa evitando abortar transacción alguna.

6. Explicar este gráfico



La transacción T1 se había confirmado antes del último checkpoint (antes de la falla), por lo que no necesita ninguna acción en particular, ya fue escrita a la base de datos.

Las transacciones T2 y T3 deben rehacerse, dado que no fueron escritas definitivamente a la base de datos, por lo haber verificado un checkpoint. Las transacciones T4 y T5 se ignoran, nunca recibieron una confirmación.

7. Como trabaja la técnica de recuperación con paginación de sombra.

Esta técnica concibe que la base de datos esté compuesta por varios bloques (páginas) de disco de tamaño fijo para fines de recuperación. En esa línea, se constituye una tabla de páginas con n entradas donde la i -ésima entrada a la misma apunta a la i -ésima página de la base de datos en el disco. Esta tabla se mantiene en memoria (si no es demasiado grande) y todas las referencias a las páginas de la base de datos en el disco pasan por la tabla de páginas. Cuando una transacción comienza a ejecutarse, la tabla de páginas actual se copia a una tabla de páginas sombra, que se guarda en disco mientras la transacción usa la tabla actual.

8. ¿Como opera la variante del protocolo 2PL conocida como Strict Two Phase Locking (S2PL)?

S2PL sigue dos reglas:

- Si una transacción quiere leer un objeto deberá pedir un shared lock de dicho objeto.
- Si una transacción quiere escribir sobre un objeto deberá pedir un exclusive lock de dicho objeto.

Todos los locks retenidos por una transacción son liberados cuando la transacción se completa (commit o rollback). Cumpliendo estas reglas, un DBMS puede entrelazar acciones entre transacciones de forma segura. Aunque ayuda a administrar transacciones a un DMBS, no alcanza para evitar deadlocks. Para ello existe una variante conservadora de S2PL en la que cada transacción antes de iniciar sus operaciones deberá pedir todos los locks que va a necesitar. Con esto, evitamos deadlocks.

Optimización

1. ¿Dónde se almacenan las estadísticas utilizadas durante el proceso de optimización por costo (en el DBMS)?

Para poder estimar de forma eficiente los parámetros necesarios para calcular costos, en el catálogo de la base de datos se mantienen histogramas, para esto hay que coleccionar dichas estadísticas de los objetos que queremos que funcionen de manera óptima.

2. Suponiendo que se ejecuta un inner join entre dos tablas (R y T). La cantidad de registros $|R| = 1000$. Cantidad de registros $|T|=1$. El criterio de junta para este inner join es un único campo llamado Field1. La tabla T

tiene una PK clustered que no incluye a Field1 en su definición. Tampoco la tabla R. Pero en la tabla R hay un índice secundario por Field1 como criterio de indexación. Dicho índice posee una selectividad de 1/1000 (es como una Unique Key) y tiene 3 niveles (multiniveles).

Cuál de los métodos de join va a ser mas eficiente en este caso?

- a. Hash Join
- b. Nested Loops
- c. Sort Merge Join
- d. Index Nested Loops

El método más eficiente es **d. Index Nested Loops**, porque aprovecha el índice secundario en **R** (con alta selectividad y 3 niveles) para buscar rápidamente los registros correspondientes al único registro de **T**. Esto minimiza el costo comparado con otros métodos, que implican escaneos o clasificaciones innecesarias.

3. ¿Qué es la selectividad de una condición atómica y por qué es importante en la optimización?

Se define cómo la razón entre la cantidad de tuplas (o registros de ahora en más) que satisfacen la condición, y el número total de registros de la tabla (relación).

La selectividad es clave para que el optimizador de la base de datos elija el método de acceso más eficiente (índices o escaneo) y ordene las operaciones para minimizar el volumen de datos procesados, mejorando el rendimiento de las consultas.

4. Supongamos que se le pide a un motor de bases de datos relacional realizar un inner join entre 3 tablas T1, T2 y T3 (con |T1|, |T2| y |T3| registros).
- a. Elaborar el peor escenario posible en términos de optimización explicando cómo se ejecutaría, y potencialmente que complejidad podría tener esta solución.
 - b. Elaborar el mejor escenario posible en términos de optimización explicando cómo se ejecutaría, y potencialmente que complejidad podría tener esta solución.

a. Peor escenario posible:

En el peor escenario, el motor de la base de datos no dispone de índices relevantes en los campos utilizados para el join y debe realizar un producto cartesiano entre las tablas antes de aplicar los filtros. Esto implica evaluar todas las combinaciones posibles entre los registros de T1, T2 y T3. Inicialmente, se generan $|T1| \times |T2|$ registros intermedios entre las primeras dos tablas, que luego se combinan con T3, resultando en $(|T1| \times |T2|) \times |T3|$ registros. En términos de complejidad, esto corresponde a $O(|T1| \times |T2| \times |T3|)$ lo que genera un alto costo de procesamiento, almacenamiento de datos intermedios y acceso a disco. Este

método, como el **Nested Loops** básico, es ineficiente porque implica analizar una gran cantidad de registros intermedios que podrían no ser relevantes.

b. Mejor escenario posible:

En el mejor escenario, las tablas cuentan con índices adecuados sobre los atributos involucrados en el join, y el motor puede utilizar algoritmos más eficientes como el **Hash Join** o el **Sort-Merge Join**. Si se utiliza un **Hash Join**, los registros de las tablas más pequeñas se dispersan en buckets mediante una función hash, y luego los registros de la tabla restante se procesan en una única pasada, comparándolos con los buckets correspondientes. Esto reduce significativamente el costo, con una complejidad aproximada de $O(|T1|+|T2|+|T3|)$, dependiendo del tamaño de las tablas y la cantidad de registros coincidentes. Adicionalmente, si las tablas tienen índices clusterizados por los atributos de unión, un **Sort-Merge Join** puede ser aún más eficiente, ya que compara directamente los registros ordenados sin necesidad de dispersiones adicionales. Este enfoque minimiza los registros intermedios y optimiza tanto el uso de memoria como los tiempos de procesamiento.

5. Describir los 3 ajustes básicos que pueden realizarse para aliviar los problemas de performance:

Existen 3 ajustes básicos que pueden hacerse para aliviar los problemas de performance:

Ajustar el consumo unitario: Usando menos recursos por transacción o reduciendo el tiempo de servicio. También pueden reducirse el número de operaciones de I/O por transacción.

Ajustar la demanda funcional: Replanificando o redistribuyendo el trabajo.

Ajustando la capacidad: Incrementando o realojando recursos.

6. Que maneras tiene el RDBMS para resolver una operación de junta (JOIN). Enumere y describa.

Nested Loops (ciclos anidados): Por fuerza bruta, por cada registro de uno de los conjuntos, se evaluarán todos los registros del otro. De ese modo la cantidad de registros a evaluar es el producto de las cantidades de registros de cada conjunto que participa en el join.

Index Nested Loops: Similar al anterior, sólo que en el caso en que se pueda utilizar un índice en alguna de las tablas participantes en el join, se accederá a esa parte de la junta vía el índice, reduciendo la complejidad.

Sort-Merge Join: Si las dos tablas del join tienen un índice clusterizado por el atributo de junta, se podrá implementar el sort-merge join de la forma más eficiente: Se examinan ambos archivos en orden, buscando los registros que tienen los mismos valores en ambas tablas, de este modo se logra un apareo por el cual un registro es examinado una sola vez para compararlo con el otro. Para los índices secundarios, si en vez de ser índices tipo cluster, nos encontramos con índices secundarios, también se puede hacer este apareo, pero teniendo en cuenta que quizás los registros

estén dispersos por muchos bloques, por lo cual puede no ser muy eficiente.

Hash-Join: Los registros de ambas tablas se dispersan al mismo archivo de dispersión, usando la misma función de hash, con los atributos de join cómo claves del hash. Una sola pasada por la tabla con el menor número de registros dispersa sus registros a buckets del archivo de dispersión. Una sola pasada por el otro archivo dispersa entonces sus registros a la cubeta apropiada, dónde el registro se combina con todos los registros coincidentes de la primera tabla.

Seguridad

1. Suponiendo que se genera la siguiente estructura de seguridad:
CREATE role MANAGER;
GRANT select, insert, update, delete on employee to MANAGER;
GRANT MANAGER to user1;
user 1 evidentemente puede hacer SELECT * FROM EMPLOYEE; Si ahora el DBA ejecuta (de manera exitosa): REVOKE select, insert, update, delete on employee FROM user1;
¿Que ocurre cuando el user1 vuelva a ejecutar “SELECT * FROM EMPLOYEE;”?
 - a. Da un error de seguridad
 - b. Devuelve siempre 0 registros de la tabla EMPLOYEE
 - c. Devuelve normalmente los datos de la tabla EMPLOYEE. Se le hace un REVOKE al user1, pero user1 nunca tuvo esos permisos, los tiene el rol. Mientras siga perteneciendo al rol y el rol tenga los permisos, va a poder ejecutar las acciones.
2. Explicar el atributo especial del comando GRANT: “WITH GRANT OPTION”
Utilizamos el comando GRANT para otorgar permisos a un usuario. El atributo “WITH GRANT OPTION” permite a los usuarios pasar la autoridad de otorgar el privilegio a otros. Puede administrarse de dos maneras:
 - Descentralizado: Fácil de establecer, más difícil de controlar. A medida que más y más usuarios obtienen la autoridad para conceder privilegios, el alcance de la autoridad se amplía y es difícil de manejar.
 - Centralizado: Es generalmente más fácil de administrar, pero carga al administrador por ser el único en manejar los privilegios en el entorno.

